Teams admin profile implementation spec

Status: implementation proposal for `dev` **Date:** 2026-10-16 **Audience:** frontend, backend, infrastructure, and QA engineers

1. Purpose

Add a discoverable admin profile for workspace Teams without creating a second Team surface or a second authorization model. The existing `/teams` routes on `dev` already provide the Team overview, Team detail, member management, ownership transfer, and Team-scoped issues. This work should finish the admin entry point and make the current behavior explicit, testable, and deployable.

This spec is code-backed against `origin/dev` at `6977ce58`. The current implementation map in `docs/core/technical.md` remains authoritative when this document conflicts with an implementation detail.

2. Scope

In scope

- Add a Teams entry under workspace administration in the existing Settings dialog.
- Deep-link that entry to the canonical `/teams` admin surface rather than duplicating Team CRUD inside Settings.
- Preserve the existing admin overview and detail designs:
  - workspace summary cards;
  - Team cards with create, edit, delete, and ownership actions;
  - Members and Issues tabs on a Team detail page;
  - member search, role/type filtering, issue search, and issue list/kanban switching.
- Keep Team membership, ownership, issue visibility, and workspace role rules enforced on the server.
- Verify the existing realtime invalidation path and deployment binding.
- Add regression coverage for the Settings entry point, authorization, query cache behavior, and Team workflows.

Out of scope

- A second Team route, database, or settings-specific API.
- Per-Team arbitrary JSON policy/configuration.
- Changing workspace roles or introducing Team roles beyond workspace role display plus Team ownership.
- Replacing the existing issue model with a Team issue model.
- App-wide websocket work unrelated to Team cache invalidation.

3. Product decisions

- Database and API terminology is `Team`; UI copy may use “Teams” or “Departments” where the design requires it.
- A Team belongs to one workspace and can contain users and agents.

- A Team has one human owner. Agents cannot own Teams.
- Workspace owners and admins can manage every Team.
- A Team owner can manage their own Team, but only workspace owners/admins can transfer ownership or delete the Team.
- A normal member can see only Teams they belong to.
- A normal Team member sees only Team issues assigned to them. The Team owner can see all issues in their Team; workspace managers can see all issues.
- UI `can_manage` and `can_transfer_owner` fields are rendering hints only. API handlers remain the authorization boundary.

4. Current `dev` surface

Surface	Current implementation
Team overview	`apps/web/src/routes/_authenticated/_app/teams.index.tsx` and `features/teams/components/team-overview.tsx`
Team detail	`apps/web/src/routes/_authenticated/_app/teams.$teamId.tsx` and `features/teams/components/team-detail.tsx`
Cross-Team issues	`apps/web/src/routes/_authenticated/_app/teams.issues.tsx` and `features/teams/components/team-work-page.tsx`
Client queries	`features/teams/queries.ts`
Client mutations	`features/teams/mutations.ts`
API client	`apps/web/src/lib/api/teams.ts`
Team service	`apps/web/src/lib/server/teams.ts`
Access control	`apps/web/src/lib/server/team-access.ts` and `workspace-permissions.ts`
Request validation	`apps/web/src/lib/server/validation/teams.ts`
Shared contracts	`packages/core/src/types/team.ts`
Database schema	`packages/db/src/schema/teams.ts` and `packages/db/src/schema/issues.ts`
Realtime invalidation	`apps/web/src/lib/server/realtime.ts` and `apps/web/src/lib/realtime/invalidation.ts`
Settings shell	`features/settings/components/settings-page.tsx` and `settings-dialog.tsx`

The Settings shell currently exposes Account, Appearance, Developer, General, and Members tabs. Its workspace section should gain a Teams entry. The current Teams overview has a disabled `Manage Teams` button; that button should either open the Settings Teams entry or be removed when the Settings link is shipped. It must not remain a dead control.

Baseline correction before extension

`features/teams/queries.ts` currently selects `data.teams` from the list response, while `team-overview.tsx` reads `unique_member_count` from the full response envelope. Resolve this contract mismatch before adding the Settings entry. Prefer preserving the response envelope in the shared query option and deriving `teams` at the component boundary.

5. Frontend implementation

5.1 Settings entry point

Extend `SettingsPage` with a workspace `Teams` item and a corresponding content placeholder only if the Settings UX requires an in-dialog landing state. The recommended behavior is a link/action that closes Settings and navigates to `/teams`, because `/teams` already owns the complete admin workflow.

Required behavior:

- Show the entry only when the active workspace membership role is `owner` or `admin`, unless product explicitly wants members to discover their visible Teams there.
- Preserve the selected Settings tab when Settings is reopened.
- Close the dialog before navigation so the Teams route is not mounted beneath an open modal.
- Keep `/teams` usable directly for existing navigation and deep links.
- Do not infer permissions from the presence of the navigation item.

Use the existing Settings dialog store and TanStack Router navigation. Do not add a React effect for synchronization; the click handler is the state transition boundary.

5.2 Canonical routes

Route	Purpose	Access
`/teams`	Admin overview or member-visible Team view	Authenticated workspace member
`/teams/$teamId`	Team detail with Members and Issues tabs	Workspace manager or Team member
`/teams/issues`	Cross-Team issue work surface	Workspace manager; otherwise only if the existing API returns visible issues

The Team detail tab remains URL state: `?tab=members` or `?tab=issues`. Invalid tab values must resolve to `members`.

5.3 Data loading and cache behavior

- Route loaders prefetch workspace-scoped Team queries through `prefetchActiveWorkspace`.
- Query keys remain workspace-scoped:
  - `['teams', workspaceId, 'list']`;
  - `['teams', workspaceId, 'detail', teamId]`;
  - `['teams', workspaceId, 'members', teamId]`;
  - `['issues', workspaceId, 'team', teamId]`.
- Preserve previous data during refetches so navigation and mutations do not flash empty states.
- After create/update/delete/member/owner mutations, invalidate only the affected Team list, detail, member, and issue keys.
- Realtime events should invalidate authoritative queries; they should not become a second client-side copy of Team state.

6. Backend implementation

6.1 API contract

The existing endpoints are the canonical contract:

Method	Endpoint	Behavior
`GET`	`/api/teams`	Return visible Teams, total, and distinct visible member count
`POST`	`/api/teams`	Create a Team and optional initial memberships transactionally
`GET`	`/api/teams/:teamId`	Return caller-relative Team detail and counts
`PATCH`	`/api/teams/:teamId`	Update name and/or description
`DELETE`	`/api/teams/:teamId`	Delete an empty Team; only workspace managers may delete
`GET`	`/api/teams/:teamId/members`	Return resolved user/agent display rows
`POST`	`/api/teams/:teamId/members`	Add one user or agent membership
`DELETE`	`/api/teams/:teamId/members/:membershipId`	Remove a membership, subject to owner/issue rules
`PUT`	`/api/teams/:teamId/owner`	Transfer ownership to a workspace user; manager only

Request bodies must use the shared Zod schemas in `apps/web/src/lib/server/validation/teams.ts`. The user/agent membership body is a discriminated union and must never accept both identity fields.

Errors must keep the existing stable `{ error, code }` envelope. Do not expose raw Postgres errors, emails in access-denied logs, or resource content in realtime payloads.

6.2 Authorization

Every handler must authenticate first and resolve the workspace from the request context. Use the existing access helpers:

- `requireTeamAccess` for reads;
- `requireTeamManage` for Team/member writes;
- `requireTeamOwnerTransfer` for ownership transfer;
- `requireWorkspacePermission` for workspace-level create/delete policy.

Do not rely on client role checks, route visibility, or `can_*` DTO fields for security. Every Team and issue query must carry workspace and Team scope in the database predicate. Composite workspace-safe foreign keys remain required.

6.3 Data model

Keep the current Drizzle model:

- `team`: workspace, name, description, human owner, creator, timestamps;
- `team_member`: exactly one `user_id` or `agent_id`, Team/workspace scope, creator, timestamp;
- `issue.team_id`: nullable, with a composite Team/workspace foreign key;
- indexes for workspace/name, Team membership, and Team/status issue reads.

If a schema change becomes necessary, edit the Drizzle schema first, then run:

```
pnpm --filter @garden/db db:generate
```

Do not hand-write generated migration SQL. Use raw SQL only for an explicit data repair or backfill that Drizzle cannot represent.

7. Realtime and deployment

The write path is:

1. authenticate and authorize;
2. write Postgres;
3. publish a workspace-scoped Team event with `waitUntil`;
4. invalidate affected TanStack Query keys;
5. refetch authoritative data.

Current `dev` has `WORKSPACE_REALTIME` bound to `WorkspaceRealtimeDO` in `apps/web/wrangler.jsonc`, with migration `v11`. Verify the same binding and Durable Object migration are present in the Alchemy target before deployment.

Prefer event payloads containing workspace ID, Team ID, membership ID, and an operation type. Do not broadcast a caller-relative `Team` DTO containing `can_manage`, `can_transfer_owner`, or visibility-dependent counts to every workspace client. Clients receiving an event should invalidate and refetch; they should not apply an untrusted partial Team object to all viewers.

Deployment checks:

```
pnpm --filter @garden/web typecheck
pnpm --filter @garden/web test
pnpm typecheck
pnpm test
pnpm build
```

For local verification:

```
pnpm --filter @garden/db db:migrate
pnpm dev
```

For the persistent dev deployment, use the repository command rather than a direct Wrangler deploy:

```
pnpm deploy:dev
```

The root scripts currently make `deploy:dev` run typecheck plus Alchemy, while `pnpm deploy` also runs `deploy:migrate`. Confirm the target database migration has been applied before deploying a schema change; do not assume the dev Alchemy step applies Postgres migrations.

8. Testing plan

Frontend

- Settings shows/hides Teams according to the active workspace role.
- Clicking Teams closes Settings and navigates to `/teams`.
- Direct `/teams` navigation still renders the same surface.
- Admins see workspace summary cards and management actions.

- Members do not see management actions or workspace-wide counts.
- Team detail tab state survives refresh and invalid values fall back safely.
- Create, edit, transfer, add member, remove member, and delete mutations invalidate the correct queries.
- The list response envelope preserves `unique_member_count`.

Extend `apps/web/src/features/teams/components/team-pages.test.tsx` and add focused Settings tests rather than coupling tests to implementation details.

Backend and integration

- unauthenticated requests return `401`;
- non-members cannot read another Team;
- workspace managers can read/manage every Team;
- Team owners can manage their Team but cannot transfer ownership or delete;
- agents cannot become owners;
- cross-workspace users and agents cannot be added;
- duplicate names and memberships return stable `409` codes;
- deleting a Team with issues returns the stable `TEAM_HAS_ISSUES` error;
- issue counts obey caller visibility;
- realtime events are emitted only after successful writes;
- realtime payloads do not expose viewer-relative fields.

Use the existing Teams integration fixtures and route tests under `apps/web/src/routes/api/teams/`.

9. Acceptance criteria

- An owner/admin can open Settings, choose Teams, and land on the existing Teams admin surface without a duplicate page or stale modal.
- A member cannot use the Settings entry or API to access Teams they do not belong to.
- All Team mutations work through the existing API and return typed, stable responses.
- Team issue visibility is unchanged and enforced server-side.
- The list response and overview agree on one response shape.
- Realtime updates invalidate affected Team views without broadcasting caller-relative permissions or counts.
- Local typecheck/tests and the documented dev deployment checks pass.

10. Follow-ups

The existing Teams follow-up issues remain separate from this entry-point work: #152, #153, #154, and #155. Do not fold unrelated performance, realtime payload, or database-error cleanup into the Settings integration unless its acceptance criteria require it. If the designs require true per-Team policy controls, write a separate contract-first spec for those policies before adding schema or UI fields.